

Osnove podatkovne znanosti



Fakultet informatike u Puli

Vježba 2

Uvod u Python

Predavanja: doc. dr. sc. Ivan Lorencin

Vježbe: Elvis Starić mag. inf.

U prethodnoj vježbi računali smo pomoću formula u tabličnom kalkulatoru. Sada ćemo podatke zapisivati i obrađivati programskim kodom. Umjesto adresa ćelija koristit ćemo imena varijabli, a postupak obrade zapisat ćemo kao niz naredbi u Pythonu.

Ova skripta namijenjena je i radu na nastavi i kasnijem samostalnom učenju. Primjere treba pokrenuti, pročitati rezultat i usporediti ga s objašnjenjem. Prije otvaranja skrivenog ispisa pokušajte predvidjeti što će program ispisati. Ako se rezultat razlikuje od očekivanoga, vratite se na vrijednosti i tipove podataka u svakom koraku.

Radimo u već pripremljenom okruženju, u Python datotekama u VS Codeu. Primjeri se pokreću na računalu; u ovoj su skripti prikazani kod i skriveni očekivani rezultati. Grananja, petlje i definiranje vlastitih funkcija obradit ćemo u sljedećoj vježbi. Sada gradimo znanje potrebno da bismo ih razumjeli.

1 Pisanje i pokretanje Python programa

1.1 Izrada i pokretanje Python datoteke

Program je zapis postupka koji računalu treba izvršiti. U ovoj vježbi pišemo jednostavne programe u kojima se naredbe izvršavaju redom, od prve prema posljednjoj. Svaki korak koristi vrijednosti dostupne u tom trenutku. Python ne pogađa što smo htjeli napisati: naziv, navodnici, operatori i njihov redoslijed imaju određeno značenje.

Program, izvorni kod i interpreter

Program je skup naredbi kojima opisujemo postupak. **Izvorni kod** je tekst tih naredbi zapisan u programskom jeziku. **Python interpreter** izvršava Python kod. Datoteka s nastavkom `.py` sadrži izvorni kod, dok se rezultat izvršavanja u našem načinu rada prikazuje u terminalu.

Postupak rada s primjerima:

1. Napravite mapu `Vježba 2` za svoje Python datoteke
2. Otvorite je u VS Codeu i napravite datoteku `primjer.py`
3. Upišite kod jednog primjera i spremite datoteku
4. Pokrenite datoteku naredbom za pokretanje Python datoteke u terminalu, kao u uvodnoj vježbi
5. Pročitajte novi ispis u terminalu i usporedite ga s očekivanim rezultatom
6. Za sljedeći primjer zamijenite sadržaj datoteke ili napravite novu datoteku s opisnim nazivom.

Primjeri u skripti s potpunim programom samostalne su cjeline. Pokretanje jednog ne zahtijeva da prethodno pokrenete drugi. Samostalne zadatke na kraju spremite u zasebne datoteke kako bi sva rješenja ostala dostupna za provjeru.

1.2 Ispis pomoću funkcije print

Funkcija `print()` prikazuje vrijednost u terminalu. Zagrade označavaju poziv funkcije, a unutar njih navodimo što želimo ispisati. Već gotove funkcije poput `print()` koristimo bez potrebe da poznamo njihov unutarnji programski kod.

Funkcija i argument

Funkcija je imenovani postupak koji možemo pozvati. **Argument** je vrijednost koju predajemo funkciji pri pozivu. U pozivu `print("Pozdrav")` tekst `"Pozdrav"` predstavlja argument funkcije `print`.

Zadatak: ispišite naziv kolegija i rezultat jednostavnog izračuna.

Postupak:

1. Upišite sljedeći program
2. Predvidite razliku između drugog i trećeg retka
3. Pokrenite program i provjerite ispis.

```
print("Osnove podatkovne znanosti")
print(12 + 8)
print("12 + 8")
```

▼ Prikaži ispis i objašnjenje

```
Osnove podatkovne znanosti
20
12 + 8
```

Bez navodnika Python izračunava izraz `12 + 8` i funkciji predaje broj 20. Unutar navodnika znakovi su tekst: Python ih ispisuje upravo kako su zapisani. Navodnici koji omeđuju tekst nisu dio ispisa.

Svaki poziv `print()` prema zadanim postavkama završava prijelazom u novi red. U jednom pozivu možemo navesti više argumenata, odvojenih zarezom. `print()` između njih prema zadanim postavkama umeće razmak.

```
print("Broj studenata:", 12)
print("Zbroj:", 12 + 8)
```

▼ Prikaži očekivani ispis

```
Broj studenata: 12
Zbroj: 20
```

U `.py` datoteci sam izraz `12 + 8` neće automatski prikazati rezultat. Za vidljivi ispis koristimo `print(12 + 8)`. To je važno ako ste negdje vidjeli primjere iz interaktivne konzole, u kojoj se vrijednost izraza može prikazati i bez poziva funkcije `print()`.

1.3 Komentari i čitljivost

Komentar počinje znakom `#` i traje do kraja retka. Namijenjen je osobi koja čita kod. Python ga ne izvršava. Prazne retke koristimo da vizualno odvojimo dijelove programa.

```
# Broj sudionika u dvjema grupama.
print(14 + 16)

print("# nije komentar unutar teksta")
```

▼ Prikaži očekivani ispis

```
30
# nije komentar unutar teksta
```

Komentar treba pojasniti svrhu ili razlog postupka. Ne treba uz svaki redak ponavljati ono što se iz koda već vidi. U ovim primjerima komentari su ponekad detaljniji jer služe učenju. Naredbe zasad pišemo od početka retka, bez nepotrebnih razmaka ispred njih. Uvlačenje koda dobit će posebno značenje kada budemo obrađivali blokove naredbi.

Pregled funkcija i metoda

Funkcija ili metoda	Namjena
<code>print()</code>	ispisuje vrijednosti i poruke u terminal.

Zadatak

Kako biste pomoću funkcije `print()` ispisali sljedeća tri retka?

PODACI O KOLEGIJU

Naziv: Osnove podatkovne znanosti

ECTS: 6

▼ Prikaži moguće rješenje

```
print("PODACI O KOLEGIJU")
print("Naziv: Osnove podatkovne znanosti")
print("ECTS:", 6)
```

2 Varijable i dodjela vrijednosti

Najprije pokušajmo riješiti mali problem bez varijabli.

Zadatak: Napišite program koji za radionicu s 12 prijavljenih studenata ispisuje broj prijavljenih, broj potpunih grupa od po tri studenta, broj preostalih studenata, ukupan broj radnih sati ako svaki student radi tri sata te ukupan broj materijala ako svaki student dobiva dva materijala. U izvještaj uključite i oznaku 3123 te broj dvorane 12.

```
print("IZVJEŠTAJ O RADIONICI")
print("Prijavljeno studenata:", 12)
print("Broj grupa po 3 studenta:", 12 // 3)
print("Preostalo studenata:", 12 % 3)
print("Ukupan broj radnih sati:", 12 * 3)
print("Broj materijala po studentu:", 2)
print("Ukupno materijala:", 12 * 2)
print("Oznaka izvještaja: 3123")
print("Broj dvorane: 12")
print("Izvještaj je dovršen.")
```

Scenarij

1. Na kojim se retcima 12 odnosi na broj prijavljenih studenata, a na kojem označava broj dvorane?
2. Što biste morali promijeniti kada bi se prijavilo još 5 studenata?
3. Biste li smjeli naredbom za zamjenu teksta promijeniti svako pojavljivanje znakova 12 u 17? Što bi se dogodilo s oznakom izvještaja 3123?

Broj možemo ponovno upisivati svaki put, ali promjena podatka tada znači traženje i mijenjanje svih njegovih zapisa. Ne smijemo pritom mijenjati 12 u retku s brojem dvorane, a masovna zamjena znakova bi 3123 pretvorila u 3143. Varijabla nam omogućuje da podatku damo ime, vrijednost zapišemo na jednom mjestu i zatim koristimo to ime.

Varijabla i dodjela

Varijabla je imenovani podatkovni element programa: njezino ime omogućuje pristup trenutačno pridruženoj vrijednosti. **Dodjelom** se vrijednost ili rezultat izraza pridružuje imenu varijable. U zapisu broj_studenata = 12 ime varijable je broj_studenata, znak = označava dodjelu, a 12 je pridružena vrijednost tipa int. Nova dodjela može postojeću vrijednost zamijeniti drugom.

Sada isti postupak zapišimo s varijablom. Prije otvaranja ispisa promijenite samo 12 u prvom retku u 17 i predvidite što će se promijeniti u drugom retku.

Možete zamisliti ime kao oznaku preko koje pristupamo podatku. Preciznije, ime upućuje na objekt koji ima svoju vrijednost i tip. Za sada je najvažnije razlikovati ime varijable od podatka s kojim je povezano.

```
broj_studenata = 12

print("IZVJEŠTAJ O RADIONICI")
print("Prijavljeno studenata:", broj_studenata)
print("Broj grupa po 3 studenta:", broj_studenata // 3)
print("Preostalo studenata:", broj_studenata % 3)
print("Ukupan broj radnih sati:", broj_studenata * 3)
print("Broj materijala po studentu:", 2)
print("Ukupno materijala:", broj_studenata * 2)
print("Oznaka izvještaja: 3123")
print("Broj dvorane: 12")
print("Izvještaj je dovršen.")
```

▼ Prikaži ispis i objašnjenje

```
IZVJEŠTAJ O RADIONICI
Prijavljeno studenata: 12
Broj grupa po 3 studenta: 4
Preostalo studenata: 0
Ukupan broj radnih sati: 36
Broj materijala po studentu: 2
Ukupno materijala: 24
Oznaka izvještaja: 3123
Broj dvorane: 12
Izvještaj je dovršen.
```

Svako pojavljivanje imena `broj_studenata` bez navodnika znači „uzmi vrijednost povezanu s ovim imenom”. Promjenom prvog retka u `broj_studenata = 17` svi izračuni automatski koriste novu vrijednost, dok oznaka izvještaja i broj dvorane ostaju nepromijenjeni. To je prednost varijable: podatak mijenjamo na jednom mjestu.

2.1 Definiranje varijable prvom dodjelom vrijednosti

U nekim programskim jezicima prije korištenja varijable deklariramo njezino ime i tip. U Pythonu ne pišemo zasebnu deklaraciju tipa prije prve dodjele. Ime uspostavljamo dodjelom, a tip pripada dodijeljenoj vrijednosti. Zato možemo odmah napisati `bodovi = 78`, bez prethodnog navođenja da su bodovi cijeli broj.

Vrijednost mora postojati prije nego što je pokušamo koristiti. Ako u novom programu napišemo samo `print(bodovi)`, a ime `bodovi` još nije definirano, dobit ćemo pogrešku `NameError`. Python ne pretpostavlja da nepoznato ime ima vrijednost nula ili prazan tekst.

2.2 Pravila imenovanja varijabli

Dobra imena opisuju podatak. `broj_dolazaka`, `cijena_proizvoda` i `ime_studenta` jasniji su od `x`, `y` i `z`, osobito kada se kasnije vratimo programu.

Pravilo	Primjer
Ime ne počinje znamenkom.	<code>zadatak_1</code> je dopušteno, <code>1_zadatak</code> nije.
U imenu nema razmaka.	Koristimo <code>broj_bodova</code> , a ne <code>broj bodova</code> .
Velika i mala slova razlikuju se.	<code>bodovi</code> i <code>Bodovi</code> različita su imena.
Ključne riječi jezika nisu imena varijabli.	Primjerice, <code>True</code> je već dio jezika.

U primjerima koristimo mala slova i donju crtu između riječi. To je uobičajen stil imenovanja u Pythonu. Python dopušta i mnoge znakove izvan engleske abecede, ali ćemo u imenima radi jednostavnijeg tipkanja koristiti `cijena`, `kolicina` i slične nazive bez dijakritičkih znakova. U tekstu koji ispisujemo slobodno koristimo hrvatska slova.

Ne koristite `print`, `str`, `int`, `float`, `input` ili `type` kao imena svojih varijabli. To nisu sve ključne riječi, ali su imena ugrađenih mogućnosti koje ćemo koristiti. Dodjelom vlastite vrijednosti takvom imenu zaklonili biste njegovu uobičajenu ulogu.

2.3 Redosljed izračuna i dodjele vrijednosti

Dodjela nije matematička tvrdnja o jednakosti. Python prvo izračuna desnu stranu, a zatim rezultat poveže s imenom lijevo od znaka `=`.

```

cijena = 4
kolicina = 3
ukupno = cijena * kolicina
print(ukupno)

kolicina = 5
print(ukupno)

ukupno = cijena * kolicina
print(ukupno)

```

▼ Prikaži ispis i objašnjenje

```

12
12
20

```

Prva dodjela varijabli `ukupno` sprema rezultat tadašnjeg izračuna, odnosno 12. Promjena varijable `kolicina` ne izvršava prethodni izraz ponovno. Tek nova naredba `ukupno = cijena * kolicina` izračunava 20.

⚠ Razlika u odnosu na tablični kalkulator

Python dodjelom `ukupno = cijena * kolicina` ne stvara formulu koja se trajno automatski ažurira kada promijenimo ulazni podatak. Izraz se izračunava kada izvršavanje dođe do tog retka. Redoslijed naredbi zato je dio značenja programa.

2.4 Promjena vrijednosti varijable

Čest je postupak uzeti postojeću vrijednost, nešto joj dodati i rezultat ponovno dodijeliti istom imenu.

```
bodovi = 70
bodovi = bodovi + 5
print(bodovi)

bodovi += 3
print(bodovi)
```

▼ Prikaži očekivani ispis

```
75
78
```

Izraz `bodovi = bodovi + 5` čitamo kao „uzmi sadašnju vrijednost, povećaj je za 5 i dodijeli rezultat imenu `bodovi`”. U radu s brojevima `bodovi += 3` kraći je zapis povećanja vrijednosti za 3. Slično možemo koristiti `--`, `*` i `/`. Takav zapis zahtijeva da je varijabli već dodijeljena početna vrijednost.

2.5 Dodjela vrijednosti jedne varijable drugoj

```
prvi_rezultat = 80
drugi_rezultat = prvi_rezultat
prvi_rezultat = 90
print(prvi_rezultat)
print(drugi_rezultat)
```

▼ Prikaži ispis i objašnjenje

```
90
80
```

Imenu `drugi_rezultat` pridružena je vrijednost 80. Kasnija dodjela 90 imenu `prvi_rezultat` ne mijenja s kojim je brojem povezano drugo ime. Ovdje radimo s brojevima; ponašanje promjenjivih složenih objekata obradit ćemo kada uvedemo takve tipove.

Zadatak

Za radionicu je prijavljeno 18 studenata. Troje je otkazalo dolazak, a zatim su se prijavila još dva studenta. Kako biste pomoću jedne varijable pratili promjene i na kraju ispisali trenutačan broj prijavljenih?

▼ Prikaži moguće rješenje i ispis

```
broj_prijavljenih = 18  
broj_prijavljenih = broj_prijavljenih - 3  
broj_prijavljenih = broj_prijavljenih + 2  
print("Trenutačno prijavljenih:", broj_prijavljenih)
```

Trenutačno prijavljenih: 17

3 Osnovni tipovi podataka

3.1 Razlika između vrijednosti i tipa podatka

Vrijednost govori koji podatak imamo, a tip određuje kako ga Python tumači i koje operacije nad njim imaju smisla. Broj 12 može se pribrajati drugim brojevima. Tekst "12" sastoji se od dva znaka i ponaša se kao tekst. To što čovjek iz teksta može pročitati broj ne znači da ga Python automatski koristi kao broj.

💡 Tip podatka

Tip podatka određuje vrstu vrijednosti i pravila rada s njom. Osnovni tipovi koje ovdje koristimo su `int` za cijele brojeve, `float` za brojeve s pomičnim zarezom, `str` za tekst i `bool` za logičke vrijednosti.

Tip	Primjeri	Primjena
<code>int</code>	<code>12</code> , <code>0</code> , <code>-3</code>	Broj dolazaka, broj proizvoda.
<code>float</code>	<code>2.5</code> , <code>0.0</code> , <code>-4.75</code>	Temperatura, masa, prosjek.
<code>str</code>	<code>"Pula"</code> , <code>"S01"</code> , <code>"12"</code>	Imena, oznake, tekstualni zapis.
<code>bool</code>	<code>True</code> , <code>False</code>	Rezultat provjere ili stanje.

3.2 Provjera tipa funkcijom type

Funkcija `type()` vraća tip vrijednosti koju joj predamo. Za prikaz tog rezultata možemo pozvati `print(type(...))`. Unutarnji poziv `type()` prvo dobiva rezultat, a vanjski `print()` prikazuje ga u terminalu.

```
broj_studenata = 12
prosjek = 73.5
oznaka = "12"
rad_predan = True

print(type(broj_studenata))
print(type(prosjek))
print(type(oznaka))
print(type(rad_predan))
```

▼ Prikaži očekivani ispis

```
<class 'int'>
<class 'float'>
<class 'str'>
<class 'bool'>
```

Zapis `<class 'int'>` ovdje nam govori da je riječ o tipu `int`. Pojam klase zasad ne trebamo dodatno obrađivati da bismo provjeravali tipove svojih podataka.

3.3 int: cijeli brojevi

Cijeli brojevi nemaju decimalni dio. Mogu biti pozitivni, negativni ili nula. Broj dolazaka 5 zapisujemo kao `5`, bez navodnika. Tip `int` primjeren je brojanju diskretnih vrijednosti, primjerice broju knjiga ili broju studenata.

Podatak koji izgleda brojčano nije nužno količina. Oznaku `"0012"` korisno je čuvati kao tekst ako su početne nule dio oznake. Zbrajanje takvih oznaka obično nema smisla. Tip biramo prema značenju podatka, a ne samo prema znakovima od kojih je sastavljen.

3.4 float: decimalne vrijednosti

Brojeve poput 2,5 u Python kodu pišemo s decimalnom točkom: `2.5`. To vrijedi neovisno o jeziku operacijskog sustava. Zapis `2,5` nije Pythonov zapis decimalnog broja i ne smije se koristiti kao zamjena za `2.5`.

```
cijeli = 5
decimalni = 5.0
print(cijeli)
print(decimalni)
print(type(cijeli))
print(type(decimalni))
```

▼ Prikaži očekivani ispis

```
5
5.0
<class 'int'>
<class 'float'>
```

Vrijednosti 5 i 5.0 brojčano su jednake, ali imaju različite tipove. Tip rezultata može ovisiti i o operaciji, a ne samo o zapisanim ulaznim vrijednostima. To ćemo vidjeti kod dijeljenja.

Pokušajte predvidjeti što će se dogoditi ako u kodu umjesto decimalne točke napišemo zarez.

```
cijena = 2,5
ukupno = cijena + 3.0
print(ukupno)
```

▼ Prikaži poruku pogreške i objašnjenje

```
TypeError: can only concatenate tuple (not "float") to tuple
```

Zapis `cijena = 2,5` ne stvara decimalni broj. Python zarez tumači kao razdvajanje dviju vrijednosti i zato `cijena` nije broj nad kojim možemo računati s `3.0`. Točnije, Python je stvorio složeni tip koji ćemo obraditi u kasnijoj vježbi. Ovdje je važno pravilo: decimalne brojeve u Pythonu uvijek zapisujemo decimalnom točkom, primjerice `cijena = 2.5`.

3.5 str: niz znakova

Tekst zapisujemo između jednostrukih ili dvostrukih navodnika. Početni i završni navodnik moraju odgovarati. `"Pula"` i `'Pula'` predstavljaju istu tekstualnu vrijednost.

```
grad = "Pula"
poruka = 'Student kaže: "Pozdrav!'"
prazan_tekst = ""

print(grad)
print(poruka)
print(len(prazan_tekst))
```

▼ Prikaži očekivani ispis

```
Pula
Student kaže: "Pozdrav!"
0
```

Vanjski jednostruki navodnici u drugoj vrijednosti omogućuju da dvostruki navodnici budu obični znakovi unutar teksta. Prazan tekst `""` postoji i ima duljinu nula. Tekst `" "` s jednim razmakom nije prazan: sadrži jedan znak. Funkciju `len()` detaljnije ćemo koristiti u poglavlju o tekstu.

```
prazan_tekst = ""
tekst_s_razmakom = " "

print("Duljina praznog teksta:", len(prazan_tekst))
print("Duljina teksta s jednim razmakom:", len(tekst_s_razmakom))
```

▼ Prikaži očekivani ispis

```
Duljina praznog teksta: 0
Duljina teksta s jednim razmakom: 1
```

3.6 bool: logičke vrijednosti

Tip `bool` ima dvije vrijednosti, `True` i `False`. Pišemo ih velikim početnim slovom i bez navodnika. Vrijednost `True` nije isto što i tekst `"True"`, jednako kao što broj 12 nije tekst `"12"`.

```
rad_predan = True
zapis_u_datoteci = "True"
print(type(rad_predan))
print(type(zapis_u_datoteci))
```

▼ Prikaži očekivani ispis

```
<class 'bool'>
<class 'str'>
```

Logičke vrijednosti koristit ćemo za provjeru podataka. Već u ovoj vježbi možemo dobiti odgovor na pitanje „je li prosjek barem 60?” bez pisanja grananja programa.

3.7 Promjena tipa podatka pri novoj dodjeli

Istom imenu u Pythonu možemo kasnije dodijeliti vrijednost drugog tipa. To se naziva dinamičkim tipiziranjem. Ne znači da tipovi nisu važni: upravo tip vrijednosti određuje što možemo učiniti s njom.

```
rezultat = 80
print(type(rezultat))
rezultat = "nije poznato"
print(type(rezultat))
```

▼ Prikaži očekivani ispis

```
<class 'int'>
<class 'str'>
```

Ovo je dopušteno, ali u stvarnom postupku obrade može otežati rad: nakon druge dodjele više ne možemo računati s varijablom kao da sadrži broj. U primjerima zadržavamo dosljednu namjenu varijabli i tekstualnim međukoracima dajemo zasebna imena.

3.8 None: odsutnost vrijednosti

Ponekad vrijednost još ne postoji ili nije poznata. Python ima posebnu vrijednost `None` kojom to možemo označiti. Ona nije broj nula, prazan tekst ni logička vrijednost `False`.

```
bodovi = None
print(bodovi)
print(type(bodovi))
```

▼ Prikaži očekivani ispis

```
None
<class 'NoneType'>
```

S vrijednošću `None` ne možemo izravno računati prosjek kao s brojem. Ovdje je upoznajemo da ne bismo nepoznate rezultate zamjenjivali nulom. Postupanje s nedostajućim podacima razradit ćemo kasnije.

Pregled funkcija i metoda

Funkcija ili metoda	Namjena
<code>type()</code>	vraća tip predane vrijednosti.
<code>len()</code>	vraća duljinu tekstualne vrijednosti.

Zadatak

Koji biste tip podatka odabrali za trajanje ispita od 90 minuta, prosječnu ocjenu 4.25, oznaku studenta 0012, podatak da je prijava aktivna i još nepoznat broj dvorane? Posebno pojasnite odabir tipa podatka za oznaku studenta.

▼ Prikaži moguće rješenje

```
trajanje_minuta = 90
prosjecna_ocjena = 4.25
oznaka_studenta = "0012"
prijava_aktivna = True
broj_dvorane = None

print(type(trajanje_minuta))
print(type(prosjecna_ocjena))
print(type(oznaka_studenta))
print(type(prijava_aktivna))
print(type(broj_dvorane))
```

Oznaka je tekst jer početne nule čine dio njezina zapisa i nad njom ne planiramo računati.

4 Brojčane operacije i izrazi

4.1 Osnovne računske operacije

Zamislite da 10 studenata trebamo podijeliti u grupe od po tri člana. Koliko potpunih grupa možemo sastaviti i koliko studenata ostaje? Taj već poznati problem koristit ćemo za upoznavanje različitih načina dijeljenja u Pythonu. Tablica i primjer zatim pokazuju kojim se operatorom zapisuje svaki od tih izračuna.

💡 Operator i izraz

Operator označava operaciju, primjerice `+` za zbrajanje. **Izraz** je zapis koji se izračunava u vrijednost. Izraz `cijena * količina` koristi dvije vrijednosti i operator množenja te daje novi rezultat.

Operator	Značenje	Primjer	Rezultat
<code>+</code>	zbrajanje	<code>10 + 3</code>	<code>13</code>
<code>-</code>	oduzimanje	<code>10 - 3</code>	<code>7</code>
<code>*</code>	množenje	<code>10 * 3</code>	<code>30</code>
<code>/</code>	dijeljenje	<code>10 / 3</code>	približno <code>3.3333</code>
<code>//</code>	cjelobrojno djeljenje	<code>10 // 3</code>	<code>3</code>
<code>%</code>	ostatak pri dijeljenju	<code>10 % 3</code>	<code>1</code>
<code>**</code>	potenciranje	<code>10 ** 3</code>	<code>1000</code>

Brojevi u izrazima mogu biti zapisani izravno ili dohvaćeni preko imena varijabli. Za izravno zapisanu vrijednost poput `3` ili `"Pula"` koristi se i naziv **literal**.

```
a = 10
b = 3
print(a + b)
print(a - b)
print(a * b)
print(a / b)
print(a // b)
print(a % b)
print(a ** b)
```

▼ Prikaži očekivani ispis

```
13
7
30
3.3333333333333335
3
1
1000
```

4.2 Redoslijed računanja i zagrade

Množenje i dijeljenje imaju prednost pred zbrajanjem i oduzimanjem. Zagrade koristimo da izričito odredimo što se računa zajedno. Razmaci poboljšavaju čitljivost, ali ne mijenjaju taj redoslijed.

```
prvi = 10
drugi = 20
print(prvi + drugi / 2)
print((prvi + drugi) / 2)
```

▼ Prikaži ispis i objašnjenje

```
20.0
15.0
```

U prvom se prvo **20** dijeli s **2**, a zatim se dodaje **10**. U drugom se prvo zbrajaju **10** i **20**, a njihov se zbroj dijeli s **2**.

⚠ Program se može izvršiti i dati pogrešan odgovor

Python provjerava može li izvršiti zapisanu naredbu, ali ne zna koje ste pitanje htjeli riješiti. Pogrešno postavljene zagrade mogu dati valjan broj koji nije traženi rezultat. Provjera smisla rezultata dio je programiranja.

Primjer: Tri proizvoda stoje po 20 eura, a na cijeli iznos odobrava se popust od 10 %. Koji izraz ispravno računa cijenu nakon popusta?

```
cijena = 20
kolicina = 3
popust = 0.10

iznos1 = cijena * kolicina - popust
iznos2 = cijena * kolicina * (1 - popust)

print("Iznos 1:", iznos1)
print("Iznos 2:", iznos2)
```

▼ Prikaži ispis i objašnjenje

Iznos 1: 59.9

Iznos 2: 54.0

Prvi izraz oduzima 0.10 eura od ukupne cijene 60 eura, iako 0.10 označava 10 %, a ne deset centi. U drugom izrazu najprije se računa faktor nakon popusta, $1 - 0.10$, odnosno 0.90. Zatim se ukupna cijena množi s tim faktorom: $60 * 0.90 = 54$ eura. Oba izraza su valjana Pythonova koda, ali samo drugi odgovara zadanom pitanju.

4.3 Dijeljenje operatorima /, // i %

Tri operatora povezana su s dijeljenjem, ali ne odgovaraju na isto pitanje.

- Operator `/` daje rezultat običnog dijeljenja
- Operator `//` daje rezultat cjelobrojnog djeljenja,
- Operator `%` daje ostatak nakon djeljenja.

4.3.1 Obično dijeljenje operatorom /

Operator `/` računa količnik i vraća vrijednost tipa `float`, čak i kada je rezultat matematički cijeli broj.

```
print(10 / 4)
print(10 / 2)
```

▼ Prikaži očekivani ispis

2.5
5.0

Izraz `10 / 4` daje 2.5 jer se deset dijeli na četiri jednaka dijela. Rezultat izraza `10 / 2` jest 5.0, a ne 5, zato što operator `/` vraća `float`.

4.3.2 Broj potpunih grupa operatorom //

Operator `//` računa koliko se puta djeljitelj u cijelosti može smjestiti u zadanu vrijednost. Za pozitivne cijele brojeve njime možemo odrediti broj potpunih grupa.

```
broj_studenata = 23
velicina_grupe = 5
potpune_grupe = broj_studenata // velicina_grupe
print("Potpune grupe:", potpune_grupe)
```

▼ Prikaži očekivani ispis i objašnjenje

Potpune grupe: 4

Od 23 studenta možemo sastaviti četiri potpune grupe od po pet članova. Petu grupu ne možemo popuniti.

Kod negativnih brojeva `//` rezultat zaokružuje prema dolje: `-7 // 2` daje `-4`. U ovoj vježbi operator uglavnom koristimo za brojenje potpunih grupa pozitivnih količina.

4.3.3 Ostatak pri dijeljenju operatorom %

Operator `%` vraća vrijednost koja ostane nakon podjele na potpune grupe.

```
broj_studenata = 23
velicina_grupe = 5
preostali = broj_studenata % velicina_grupe
print("Preostali studenti:", preostali)
```

▼ Prikaži očekivani ispis i objašnjenje

Preostali studenti: 3

Četiri potpune grupe obuhvaćaju 20 studenata, pa nakon podjele ostaju još tri studenta. Ako je ostatak nula, vrijednost je djeljiva bez ostatka. Primjerice, `20 % 5` daje `0`.

4.4 Preciznost decimalnih brojeva i zaokruživanje

Tip `float` većinu decimalnih razlomaka pohranjuje približno u binarnom obliku. Zbog toga rezultat nekih računa može sadržavati neočekivane posljednje znamenke.

```
rezultat = 0.1 + 0.2
print(rezultat)
print(round(rezultat, 2))
```

▼ Prikaži očekivani ispis

0.30000000000000004
0.3

Funkcija `round(vrijednost, broj_decimala)` vraća zaokruženu brojčanu vrijednost. Ne mijenja automatski originalnu varijablu. Ako rezultat želimo zadržati, dodijelimo ga imenu, primjerice `zaokruzeno = round(rezultat, 2)`.

Funkcija `round()` zaokružuje **vrijednost broja**, ali ne određuje koliko će se znamenki prikazati nakon decimalne točke. Primjerice, `round(3.5, 2)` daje broj `3.5` jer dodatna nula ne mijenja njegovu vrijednost. Ako želimo da se u ispisu uvijek vide dvije decimale, primjerice `3.50`, moramo oblikovati ispis. Oblikovanje broja obradit ćemo u poglavlju o f-stringovima.

Pregled funkcija i metoda

Funkcija ili metoda**Namjena**`round()`

vraća broj zaokružen na zadani broj decimalnih mjesta.

i Zadatak

U svrhu istraživanja prikupljeno je 47 upitnika. U jednu arhivsku mapu stane osam upitnika. Od 47 upitnika 35 je valjano. Kako biste izračunali broj potpuno popunjenih mapa, broj preostalih upitnika i postotak valjanih upitnika?

▼ Prikaži moguće rješenje i ispis

```
broj_upitnika = 47
kapacitet_mape = 8
valjani_upitnici = 35

potpune_mape = broj_upitnika // kapacitet_mape
preostali_upitnici = broj_upitnika % kapacitet_mape
postotak_valjanih = valjani_upitnici / broj_upitnika * 100

print("Potpune mape:", potpune_mape)
print("Preostali upitnici:", preostali_upitnici)
print("Postotak valjanih:", round(postotak_valjanih, 2))
```

Potpune mape: 5

Preostali upitnici: 7

Postotak valjanih: 74.47

5 Pretvorba tipova podataka

5.1 Razlika između zbrajanja brojeva i spajanja teksta

Naizgled su sljedeća dva retka gotovo jednaka. Prije nego pokrenemo, hoće li ispis biti jednak ili različit i zašto.

```
print(12 + 8)
print("12" + "8")
```

▼ Prikaži očekivani ispis

```
20
128
```

Za dva broja `+` znači zbrajanje. Za dva teksta znači spajanje. Zato pri neočekivanom rezultatu prvo provjeravamo tip ulaznih podataka. Python neće automatski pretvoriti tekst u broj samo zato što njegov sadržaj izgleda brojčano.

💡 Pretvorba tipa

Pretvorba tipa stvara vrijednost drugog tipa kada sadržaj to dopušta. `int()` pretvara prikladnu vrijednost u cijeli broj, `float()` u decimalni broj, a `str()` u tekst. Pretvorba nije automatski dokaz da je podatak smislen ili ispravno unesen.

5.2 Pretvorba tipova funkcijama int, float i str

Pogledajte vrijednost `"78"`. Čovjek je može pročitati kao broj, ali Python je vidi kao tekst. Prije primjera procijenite koji će redak dati broj `80`, a koji će ispisati tekst `Bodovi: 78`.

```
tekst_bodova = "78"
bodovi = int(tekst_bodova)
tekst_cijene = "2.50"
cijena = float(tekst_cijene)

print(bodovi + 2)
print(cijena * 4)
print("Bodovi: " + str(bodovi))
print(type(tekst_bodova))
print(type(bodovi))
```

▼ Prikaži očekivani ispis

```
80
10.0
Bodovi: 78
<class 'str'>
<class 'int'>
```

Varijabla `tekst_bodova` i dalje sadrži tekst. Funkcija `int()` nije promijenila taj tekst, nego je vratila cijeli broj koji smo pridružili drugom imenu. U trećem ispisu `str(bodovi)` omogućuje spajanje dviju tekstualnih vrijednosti. Jednostavnije bismo mogli napisati i `print("Bodovi:", bodovi)`, jer `print()` prihvaća odvojene argumente različitih tipova.

5.3 Pogreške pri pretvorbi tipova podataka

Pretvorba je moguća samo kada se zadana vrijednost može zapisati u traženom tipu. Python pritom ne pokušava pogoditi što smo namjeravali niti automatski ispravlja drukčiji zapis podatka. Ako sadržaj nije prikladan za traženu pretvorbu, program se zaustavlja i javlja pogrešku `ValueError`. Neke pretvorbe ipak uspiju, ali pritom mogu izgubiti dio podatka, pa nije dovoljno provjeriti samo je li se program izvršio.

Sljedeća tablica prikazuje nekoliko uspješnih i neuspješnih pretvorbi:

Izraz	Što se događa?
<code>int("12")</code>	Vraća cijeli broj 12.
<code>float("12.5")</code>	Vraća decimalni broj 12.5.
<code>int("12.5")</code>	<code>ValueError</code> : tekst nije zapis cijelog broja.
<code>float("12,5")</code>	<code>ValueError</code> : decimalni zarez nije prihvaćen.
<code>int("dvanaest")</code>	<code>ValueError</code> : riječ se ne prevodi u broj.
<code>int(12.9)</code>	Vraća 12 odbacivanjem decimalnog dijela.
<code>int(-12.9)</code>	Vraća -12, odnosno odbacuje decimalni dio prema nuli.

`int()` nije funkcija za zaokruživanje na najbliži cijeli broj. Ne treba je koristiti kako bismo prikrili neočekivane decimale u podatku koji mora biti cijeli broj. Primjerice, broj studenata 12,9 prije svega zahtijeva provjeru izvora podatka.

5.4 Pretvorba u logičku vrijednost

Funkcija `bool()` također postoji, ali ne razumije riječi poput „da” i „ne”.

Za tekst provjerava je li prazan. Prazan tekst daje `False`, dok svaki neprazan tekst, uključujući `"False"` i `"Ne"`, daje `True`.

Kod brojevanih vrijednosti nula daje `False`, neovisno o tome je li zapisana kao `0` ili `0.0`. Svaki broj različit od nule daje `True`, uključujući negativne brojeve.

Funkcija `bool()` zato ne provjerava je li broj pozitivan, nego samo razlikuje nulu od vrijednosti različitih od nule.

```
print(bool(""))
print(bool("Ne"))
print(bool("False"))
print(bool(0))
print(bool(5))
```

▼ Prikaži očekivani ispis

False
True
True
False
True

Za pretvaranje odgovora `"Da"` u odluku koristit ćemo usporedbu sa željenim tekstom. To ćemo pokazati nakon obrade nizova znakova. Važno je razlikovati „tekst postoji” od „sadržaj teksta znači potvrđan odgovor”.

Pregled funkcija i metoda

Funkcija ili metoda	Namjena
<code>int()</code>	pretvara prikladnu vrijednost u cijeli broj.
<code>float()</code>	pretvara prikladnu vrijednost u decimalni broj.
<code>str()</code>	stvara tekstualni prikaz vrijednosti.
<code>bool()</code>	pretvara vrijednost u logičku vrijednost prema pravilima istinitosti.

Zadatak

Cijena proizvoda zapisana je kao tekst `"4.50"`, a količina kao tekst `"3"`. Kako biste pripremili te vrijednosti za računanje i izračunali ukupnu cijenu?

▼ Prikaži moguće rješenje i ispis

```
cijena_tekst = "4.50"  
kolicina_tekst = "3"  
  
cijena = float(cijena_tekst)  
kolicina = int(kolicina_tekst)  
ukupno = cijena * kolicina  
  
print("Ukupna cijena:", ukupno)
```

Ukupna cijena: 13.5

6 Unos vrijednosti pomoću input

6.1 Unos teksta funkcijom input

Dosad smo vrijednosti zapisivali u samom kodu. Funkcija `input()` omogućuje unos tijekom izvođenja programa. Program prikaže poruku, čeka unos u terminalu i nakon pritiska tipke Enter vraća upisani tekst. Završni Enter nije dio tog teksta.

Postupak:

1. Upišite program u Python datoteku i pokrenite je
2. Nakon pitanja kliknite u terminal i unesite `Ana`
3. Pritisnite Enter i provjerite odgovor programa.

```
ime = input("Kako se zovete? ")  
print("Pozdrav,", ime)
```

▼ Prikaži primjer razgovora s programom

```
Kako se zovete? Ana  
Pozdrav, Ana
```

Tekst `Ana` u prvom retku upisuje korisnik. Drugi redak ispisuje program. Za drugo uneseno ime rezultat će se odgovarajuće promijeniti.

6.2 Pretvorba korisničkog unosa u broj

Čak i ako upišemo samo znamenke, `input()` uvijek vraća `str`. Prije računanja moramo provesti odgovarajuću pretvorbu.

```
unos = input("Broj dolazaka: ")  
broj_dolazaka = int(unos)  
print("Nakon sljedećeg dolaska:", broj_dolazaka + 1)
```

▼ Prikaži primjer za unos 4

```
Broj dolazaka: 4  
Nakon sljedećeg dolaska: 5
```

Odvojeni koraci omogućuju da provjerimo `type(unos)` i `type(broj_dolazaka)`. Kada razumijemo redoslijed, isti postupak možemo zapisati kraće:

```
broj_dolazaka = int(input("Broj dolazaka: "))
```

Najprije se izvrši unutarnji poziv `input()`, a zatim se njegov rezultat predaje funkciji `int()`.

Za decimalne brojeve koristimo `float()` i pri unosu decimalnu točku. Ako u ovom primjeru unesemo riječ ili decimalni broj umjesto cijelog broja, pretvorba će javiti pogrešku. U ovoj vježbi pretpostavljamo valjan brojčani unos. Provjeru unosa i postupanje pri pogreškama uvodit ćemo nakon grananja i drugih potrebnih koncepata.

Pregled funkcija i metoda

Funkcija ili metoda	Namjena
<code>input()</code>	prikazuje poruku i vraća korisnički unos kao <code>str</code> .

Zadatak

Kako biste od korisnika zatražili zaseban unos broja sati i broja minuta te izračunali ukupno trajanje u minutama? Pretpostavite da korisnik unosi cijele brojeve.

▼ Prikaži moguće rješenje i primjer unosa

```
sati = int(input("Broj sati: "))
minute = int(input("Broj minuta: "))
ukupno_minuta = sati * 60 + minute
print("Ukupno trajanje:", ukupno_minuta, "minuta")
```

Broj sati: 1

Broj minuta: 25

Ukupno trajanje: 85 minuta

7 Rad s tekstom

7.1 Spajanje i ponavljanje teksta

Prije pokretanja pokušajte predvidjeti što će ispisati svaki od triju poziva funkcije `print()`. Obratite pozornost na razmak između imena i prezimena te na broj ponavljanja tekstova `"-"` i `"ha"`.

```
ime = "Ana"
prezime = "Anić"
puno_ime = ime + " " + prezime
print(puno_ime)
print("-" * 10)
print("ha" * 3)
```

▼ Prikaži očekivani ispis

```
Ana Anić
-----
hahaha
```

Operator `+` spaja dva niza znakova. Ne dodaje razmak automatski, pa razmak ili interpunkciju moramo uključiti u jedan od tekstova.

Operator `*` između teksta i cijelog broja ponavlja tekst navedeni broj puta.

Zapis `"Bodovi: " + 78` nije dopušten jer spaja tekst i broj operatorom koji ovdje zahtijeva dva teksta. Dobit ćemo `TypeError`. Mogućnosti su pretvoriti broj s `str()`, predati dvije vrijednosti funkciji `print()` ili koristiti oblikovani tekst koji uvodimo kasnije.

7.2 Duljina teksta

Tekst je uređeni niz znakova, a funkcija `len()` omogućuje nam da odredimo njegovu duljinu. Tekst čiju duljinu želimo saznati predajemo funkciji kao argument, primjerice `len("Pula")`. Rezultat je cijeli broj tipa `int`, pa ga možemo ispisati, spremiti u varijablu ili kasnije koristiti u izračunima i provjerama. Funkcija pritom ne mijenja izvorni tekst.

Broji se svaki znak koji je dio tekstualne vrijednosti: slova, znamenke, razmaci i interpunkcijski znakovi. Navodnici kojima je tekst omeđen u Python kodu ne pripadaju vrijednosti i zato se ne broje. Tako tekst `"Pula"` ima duljinu 4, prazan tekst `""` duljinu 0, a tekst `" "`, koji sadrži jedan razmak, duljinu 1.

`len()` je posebno koristan kada trebamo provjeriti je li tekst prazan, utvrditi duljinu oznake ili pronaći položaj posljednjeg znaka.

```
grad = "Pula"
poruka = "Bok, Ana!"
print(len(grad))
print(len(poruka))
print(len(""))
print(len(" "))
```

▼ Prikaži očekivani ispis

4
9
0
1

⚠ Pripazite pri brojanju znakova

`len()` ne broji riječi. U tekstu `"Bok, Ana!"` broji i zarez, razmak i uskličnik. Kod nekih složenijih Unicode zapisa, primjerice pojedinih emojija ili kombiniranih slova, jedan vizualni znak može biti sastavljen od više kodnih točaka, pa se broj koji vidimo i rezultat `len()` ne moraju poklapati.

7.3 Indeksi: pristup jednom znaku

Svaki znak teksta ima položaj, odnosno indeks. Kada brojimo s početka teksta, indeksiranje počinje od nule: prvi znak ima indeks `0`, drugi indeks `1` i tako dalje. U tekstu `"Python"` prvo slovo P nalazi se na indeksu `0`, a posljednje slovo n na indeksu `5`.

Python omogućuje i pristup znakovima brojanjem od kraja teksta pomoću negativnih indeksa. Posljednji znak ima indeks `-1`, pretposljednji `-2` i tako dalje. Negativni indeksi korisni su kada trebamo znak pri kraju teksta, a ne želimo prethodno računati njegovu duljinu. Pozitivni i negativni indeks u tablici mogu se odnositi na isti znak.

Znak	P	y	t	h	o	n
Indeks	0	1	2	3	4	5
Negativni indeks	-6	-5	-4	-3	-2	-1

Uglate zagrade koristimo za pristup znaku na položaju.

⚠ Pripazite na vrstu zagrada

Uglate zagrade nisu zagrade za poziv funkcije: `len(tekst)` je poziv funkcije, a `tekst[0]` dohvat znaka.

```
tekst = "Python"
print(tekst[0])
print(tekst[2])
print(tekst[-1])
print(tekst[-2])
```

▼ Prikaži očekivani ispis

P
t
n
o

Negativni indeksi broje od kraja: `-1` je posljednji znak. Za tekst duljine `6` indeks `6` ne postoji. Pokušaj `tekst[6]` završio bi s `IndexError`. Posljednji nenegativni indeks je `len(tekst) - 1`, ali samo ako tekst **nije prazan**. Rezultat dohvaćanja jednog znaka također je `str`. Python nema zaseban osnovni tip za jedno slovo (`char` u drugom jezicima).

7.4 Izdvajanje dijela teksta

U prethodnom smo poglavlju pomoću indeksa izdvojili jedan znak. U praksi često trebamo izdvojiti cijeli dio teksta, primjerice godinu iz oznake `"ST-2026-015"` ili prva dva znaka koji određuju vrstu zapisa. Dohvaćanje svakog znaka zasebno bilo bi nepraktično, posebno kada se traženi dio sastoji od više uzastopnih znakova.

Python zato omogućuje izdvajanje raspona znakova, odnosno **rezanje** (*slicing*). Potrebno je navesti gdje željeni dio počinje i gdje završava.

💡 Rezanje niza znakova

Zapis `tekst[pocetak:kraj]` izdvaja dio teksta od početnog indeksa do završnog indeksa, pri čemu završni indeks **nije uključen**. Izvorni tekst ostaje isti, a izraz vraća novi niz znakova.

```
oznaka = "ST-2026-015"
print(oznaka[0:2])
print(oznaka[3:7])
print(oznaka[-3:])
print(oznaka[:2])
```

▼ Prikaži očekivani ispis

ST
2026
015
ST

Raspon `[3:7]` obuhvaća znakove na indeksima 3, 4, 5 i 6. Izostavljen početak znači „od početka”, a izostavljen kraj „do kraja”. Za razliku od raspona ćelija u prethodnoj vježbi, ovdje završna granica nije uključena. Rezanje se oslanja isključivo na položaj znakova i ne zna da u ovom primjeru tražimo godinu. Izraz `oznaka[3:7]` uvijek uzima znakove na indeksima od 3 do 6. Ako neka oznaka ima drukčiji format ili joj nedostaje dio zapisa, rezultat možda više neće biti godina, iako Python pritom ne mora javiti pogrešku.

Moguće je navesti i korak u obliku `tekst[pocetak:kraj:korak]`. Primjerice, `"abcdef"[:2]` daje `"ace"`. Zasad su nam najvažniji početak i kraj. Razumijevanje tih granica važnije je od pamćenja mnogo kratkih zapisa.

7.5 Posebni znakovi u tekstu

Tekst ne mora sadržavati samo vidljiva slova, znamenke i interpunkciju. Ponekad unutar iste tekstualne vrijednosti želimo označiti prijelaz u novi red, tabulator ili znak koji bi Python inače protumačio kao dio zapisa koda. Takve znakove zapisujemo pomoću **posebnih znakova**, odnosno izlaznih sekvenci (*escape sequences*).

Izlazna sekvenca počinje obrnutom kosom crtom `\`. Python obrnutu kosu crtu i znak iza nje zajedno tumači kao jednu posebnu uputu. Primjerice, zapis `\n` u kodu ne ispisuje znakove `\` i `n`, nego premješta nastavak teksta u novi red.

Zapis u kodu	Značenje u tekstu
<code>\n</code>	Prijelaz u novi red.
<code>\t</code>	Tabulator.
<code>\\</code>	Jedna obrnuta kosa crta.
<code>\"</code>	Dvostruki navodnik unutar teksta omeđenog dvostrukim navodnicima.

Dvostruka obrnuta kosa crta potrebna je kada želimo da se u samom tekstu pojavi znak `\`, a ne da započne izlaznu sekvencu.

```
poruka = "Prvi red\nDrugi red"
print(poruka)
print("Znak obrnute kose crte: \\")
```

▼ Prikaži očekivani ispis

```
Prvi red
Drugi red
Znak obrnute kose crte: \
```

Novi red unutar vrijednosti i novi red koji `print()` dodaje na kraju ispisa dvije su različite stvari. Navodnici i posebni znakovi dio su pravila zapisivanja teksta u kodu, pa pogrešan zapis može promijeniti sadržaj ili uzrokovati sintaksnu pogrešku.

Zadatak

Telefonski broj zapisan je u obliku `" +385-91-123-4567 "`. Izdvojite međunarodni pozivni broj `" +385 "`, oznaku mreže `"91"` i posljednje četiri znamenke. Zatim izradite novi zapis u kojem je srednji dio broja skriven znakovima `***`, tako da rezultat glasi `" +385-91-***-4567 "`.

▼ Prikaži moguće rješenje i ispis

```
telefonski_broj = "+385-91-123-4567"
pozivni_broj = telefonski_broj[:4]
oznaka_mreze = telefonski_broj[5:7]
zavrsne_znamenke = telefonski_broj[-4:]
skriveni_broj = telefonski_broj[:8] + "***-" + zavrsne_znamenke

print("Pozivni broj:", pozivni_broj)
print("Oznaka mreže:", oznaka_mreze)
print("Posljednje četiri znamenke:", zavrsne_znamenke)
print("Skriveni broj:", skriveni_broj)
```

Pozivni broj: +385

Oznaka mreže: 91

Posljednje četiri znamenke: 4567

Skriveni broj: +385-91-***-4567

8 Metode za obradu nizova znakova

8.1 Razlika između funkcije i metode

Dosad smo koristili funkcije poput `print()`, `len()` i `type()`. Funkciju pozivamo njezinim imenom, a vrijednost nad kojom treba raditi najčešće joj predajemo unutar zagrada. Primjerice, u zapisu `len(tekst)` funkciji `len()` predajemo varijablu `tekst` kao argument.

Neke su operacije usko povezane s određenim tipom podatka. Takve operacije u Pythonu često koristimo kao **metode**. Metodu pozivamo preko konkretne vrijednosti ili varijable: najprije navodimo objekt nad kojim radimo, zatim točku, ime metode i zagrade. U zapisu `tekst.lower()` dio `tekst` određuje nad kojom vrijednošću radimo, `lower` je ime metode, a zagrade označavaju njezin poziv.

Metode koje su dostupne ovise o tipu podatka. Tekst ima metode poput `lower()`, `strip()` i `replace()`, dok broj nema iste tekstualne metode. Zagrade su potrebne i kada metodi ne predajemo dodatne argumente, jer `tekst.lower` i `tekst.lower()` nemaju isto značenje.

Metoda

Metoda je postupak povezan s određenom vrstom objekta. U pozivu `tekst.replace("a", "b")` metodu `replace` pozivamo nad vrijednošću varijable `tekst`, a dva argumenta određuju što zamjenjujemo i čime.

8.2 lower i upper: velika i mala slova

Isti tekst može biti zapisan različitim kombinacijama velikih i malih slova, primjerice `"PuLa"`, `"PULA"` i `"pula"`. Python te zapise smatra različitim tekstualnim vrijednostima. Kada veličina slova nije važna, podatke možemo ujednačiti prije usporedbe ili daljnje obrade.

Metoda `lower()` stvara tekst u kojem su slova pretvorena u mala, a `upper()` tekst u kojem su slova pretvorena u velika. Obje metode vraćaju novu vrijednost i ne mijenjaju izvorni tekst.

```
grad = "PuLa"
mala_slova = grad.lower()
velika_slova = grad.upper()
print(mala_slova)
print(velika_slova)
print(grad)
```

▼ Prikaži očekivani ispis

```
pula
PULA
PuLa
```

Metoda `lower()` vraća tekst s malim slovima, a `upper()` s velikim. Varijabla `grad` i dalje sadrži originalni tekst. To je korisno kada želimo sačuvati izvorni zapis, a za uspoređivanje koristiti ujednačenu verziju.

Postoje i metode `capitalize()` i `title()`. Metoda `capitalize()` pretvara prvi znak cijelog teksta u veliko slovo, a ostala slova u mala. Metoda `title()` pretvara prvo slovo svake riječi u veliko, a ostala slova tih riječi u mala. Primjerice, `"pula film festival".title()` daje `"Pula Film Festival"`.

Te metode nisu zamjena za provjeru pravopisa ni pouzdan način popravljjanja svih osobnih imena i kratica. Primjerice, pretvaranje kratice `FIPU` u naslovni oblik izgubilo bi izvorni zapis kratice.

8.3 strip: uklanjanje rubnih razmaka

Pri unosu se često pojave razmaci na početku ili kraju vrijednosti. Metoda `strip()` bez argumenata uklanja rubne bjeline, uključujući razmake, tabulatore i nove retke. Unutarnje razmake ostavlja.

```
izvorno = "  Ana  Anić  "
uredjeno = izvorno.strip()
print "[" + izvorno + "]"
print "[" + uredjeno + "]"
```

▼ Prikaži očekivani ispis

```
[ Ana  Anić ]
[Ana  Anić]
```

Uglate zagrade u ispisu dodali smo da se jasno vide granice teksta. Dva razmaka između imena i prezimena ostaju. `strip()` zato nije postupak za uklanjanje svih razmaka. Postoje i `lstrip()` za lijevi te `rstrip()` za desni rub.

8.4 replace: zamjena dijela teksta

Kada podatak ima poznat, ali nepoželjan zapis, dio teksta možemo zamijeniti drugim tekstom. Primjerice, razmake u strukturiranoj oznaci možemo zamijeniti crticama ili decimalni zarez točkom prije pretvorbe u broj.

Metoda `replace(staro, novo)` prima dva argumenta: dio teksta koji tražimo i tekst kojim ga želimo zamijeniti. Vraća novi tekst u kojem su, prema zadanim postavkama, zamijenjena sva pronađena pojavljivanja. Pretraživanje razlikuje velika i mala slova, pa `"EUR"` i `"eur"` nisu ista tražena vrijednost.

```
oznaka = "ST 2026 015"
uredjena_oznaka = oznaka.replace(" ", "-")
print(uredjena_oznaka)
print(oznaka)
```

▼ Prikaži očekivani ispis

```
ST-2026-015
ST 2026 015
```

Zamjena radi sa znakovima i ne poznaje značenje podataka. Ne treba globalno uklanjati razmake iz svih tekstualnih vrijednosti: razmak u imenu proizvoda može biti važan. Prije promjene treba znati koji je očekivani format.

8.5 Spremanje rezultata tekstualnih metoda

U prethodnim se primjerima izvorna vrijednost nije promijenila nakon poziva metode. To je važno svojstvo tipa `str` i česta greška: metoda se izvrši, ali rezultat nestane ako ga nismo spremili ili odmah upotrijebili.

⚠ Nizovi znakova su nepromjenjivi

Vrijednost tipa `str` ne mijenjamo na mjestu. Metode poput `strip()`, `lower()` i `replace()` vraćaju novu vrijednost. Ako želimo dalje koristiti rezultat, moramo ga dodijeliti varijabli ili neposredno proslijediti sljedećem postupku.

```
grad = " PULA "  
grad.strip()  
print("[" + grad + "]")  
  
grad = grad.strip()  
print("[" + grad + "]")
```

▼ Prikaži ispis i objašnjenje

```
[ PULA ]  
[PULA]
```

Prvi poziv izračuna novi tekst, ali ga nigdje ne sačuva. U drugom pozivu novo značenje imena `grad` postaje dobiveni rezultat. Promijenili smo vezu imena s vrijednošću, a ne sadržaj starog tekstualnog objekta.

Isto tako, naredba `grad[0] = "p"` nije dopuštena. Dio teksta ne možemo promijeniti dodjelom jednom indeksu. Koristimo postupak koji vraća novi tekst.

8.6 Povezivanje više metoda

Više koraka može se napisati odvojeno. To je pregledno za učenje i provjeravanje međurezultata.

```
izvorno = " PuLa "  
bez_rubnih_razmaka = izvorno.strip()  
ujednaceno = bez_rubnih_razmaka.lower()  
print(ujednaceno)
```

▼ Prikaži očekivani ispis

```
pula
```

Isti konačni rezultat daje

```
ujednaceno = izvorno.strip().lower()
```

Najprije se poziva `strip()` nad izvornim tekstom, a zatim `lower()` nad dobivenim tekstom. Takvo povezivanje radi jer prvi korak vraća `str`, nad kojim je drugi korak definiran. Nakon metode koja vraća broj ne možemo samo nastaviti tekstualnom metodom kao da je rezultat i dalje tekst.

8.7 Pretraživanje teksta i provjera njegova sadržaja

Dosadašnje metode uglavnom su stvarale novu, izmijenjenu tekstualnu vrijednost. Međutim, metode možemo koristiti i za dobivanje informacija o postojećem tekstu: koliko se puta neki dio pojavljuje, na kojem položaju započinje ili zadovoljava li tekst određeno pravilo.

Rezultat takve metode nije nužno `str`. Metode `count()` i `find()` vraćaju cijeli broj, dok `startswith()`, `endswith()` i `isdigit()` vraćaju logičku vrijednost `True` ili `False`. Poznavanje tipa rezultata određuje kako ga možemo dalje koristiti.

Metoda	Što vraća?
<code>count(dio)</code>	Broj nepreklapajućih pojavljivanja dijela teksta.
<code>find(dio)</code>	Indeks prvog pojavljivanja ili -1 ako ga nema.
<code>startswith(dio)</code>	Je li tekst započeo navedenim dijelom.
<code>endswith(dio)</code>	Je li tekst završio navedenim dijelom.
<code>isdigit()</code>	Je li tekst neprazan i sastavljen od znamenki.

```
datoteka = "rezultati_2026.csv"
print(datoteka.count("_"))
print(datoteka.find("2026"))
print(datoteka.find("2025"))
print(datoteka.startswith("rezultati"))
print(datoteka.endswith(".csv"))
print("015".isdigit())
print("2.5".isdigit())
```

▼ Prikaži očekivani ispis

```
1
10
-1
True
True
True
False
```

`find()` vraća položaj, ne potvrdu prisutnosti. Ako traženi dio počinje na prvom znaku, rezultat je `0`, ako ga nema, rezultat je `-1`. Ako trebamo samo odgovor je li dio prisutan, jasnije je koristiti operator `in`, koji slijedi u poglavlju o usporedbama.

Metoda `isdigit()` nije opća provjera može li se tekst pretvoriti u `float`. Decimalna točka i minus nisu znamenke, pa `"2.5"` i `"-3"` daju `False`. Osim toga, metoda prepoznaje i neke Unicode znamenke koje nisu prihvatljiv zapis za `int()`.

8.8 Primjer: uređenje zapisa cijene

Obrada stvarnih podataka često zahtijeva više uzastopnih koraka. Prije pretvorbe tekstualne cijene u broj moramo ukloniti dijelove koji nisu dio brojčane vrijednosti i prilagoditi decimalni zapis Pythonu. Redoslijed je važan: svaka metoda prima rezultat prethodnog koraka.

U prvom prikazu koristimo zasebne varijable za međurezultate. Tako možemo provjeriti što se dogodilo nakon svakog koraka i lakše pronaći mjesto eventualne pogreške.

Zadatak: poznati tekstualni zapis cijene `" 12,50 EUR "` pretvorite u broj s kojim možemo računati.

Postupak:

1. Spremite izvorni tekst
2. Uklonite rubne razmake
3. Uklonite oznaku valute prema poznatom obliku zapisa
4. Zamijenite decimalni zarez točkom
5. Pretvorite dobiveni tekst u decimalni broj
6. Provjerite vrijednost, tip i izračun za dva proizvoda.

▼ Prikaži kod, očekivani ispis i međukorake

```
izvorna_cijena = " 12,50 EUR "  
bez_razmaka = izvorna_cijena.strip()  
bez_valute = bez_razmaka.replace(" EUR", "")  
zapis_za_python = bez_valute.replace(",", ".")  
cijena = float(zapis_za_python)  
  
print(zapis_za_python)  
print(cijena)  
print(type(cijena))  
print("Dva proizvoda:", cijena * 2)
```

12.50

12.5

<class 'float'>

Dva proizvoda: 25.0

Nakon `strip()` ostaje `"12,50 EUR"`. Nakon uklanjanja valute ostaje `"12,50"`. Zamjena zareza daje `"12.50"`, što je još tekst. Tek poziv `float()` daje broj.

Ovaj postupak vrijedi za zadani poznati format, s velikim slovima `EUR` i bez separatora tisućica. Za tekst `"1.234,50 EUR"` ista zamjena dala bi dva decimalna znaka i pretvorba ne bi uspjela. Obrada podataka traži pravila o očekivanom zapisu, a ne nasumično nizanje zamjena.

Pregled funkcija i metoda

Funkcija ili metoda	Namjena
<code>lower()</code> i <code>upper()</code>	vraćaju tekst s malim odnosno velikim slovima.
<code>capitalize()</code>	povećava prvi znak teksta, a ostala slova pretvara u mala.
<code>title()</code>	povećava prvo slovo svake riječi, a ostala slova pretvara u mala.
<code>strip()</code> , <code>lstrip()</code> i <code>rstrip()</code>	uklanjaju bjeline s oba, lijevog ili desnog ruba teksta.
<code>replace()</code>	vraća tekst u kojem je zadani dio zamijenjen drugim tekstom.
<code>count()</code>	vraća broj pojavljivanja zadanog dijela teksta.
<code>find()</code>	vraća indeks prvog pojavljivanja ili <code>-1</code> ako ga nema.
<code>startswith()</code> i <code>endswith()</code>	provjeravaju početak odnosno završetak teksta.
<code>isdigit()</code>	provjerava je li neprazan tekst sastavljen od znamenki.

Zadatak

Zadani su neujednačeni tekstualni zapisi:

```
izvorna_oznaka = " st 2026 015 "  
izvorno_ime = " aNA aNIĆ "  
izvorna_jedinicna_cijena = " 12,50 EUR "  
izvorna_datoteka = " REZULTATI_2026.CSV "
```

Pripremite nove vrijednosti tako da oznaka glasi `"ST-2026-015"`, ime `"Ana Anić"`, jedinična cijena jednog proizvoda postane broj `12.5`, a naziv datoteke `"rezultati-2026.csv"`. Zatim izračunajte ukupan iznos za tri proizvoda s tom jediničnom cijenom. Provjerite koliko crtica sadrži uređena oznaka, jesu li njezina posljednja tri znaka znamenke, počinje li naziv datoteke tekstem `"rezultati"`, završava li nastavkom `".csv"` i na kojem položaju počinje godina `"2026"`. Izvorne vrijednosti moraju ostati sačuvane.

▼ Prikaži moguće rješenje i ispis

```

izvorna_oznaka = " st 2026 015 "
izvorno_ime = " aNA aNIĆ "
izvorna_jedinicna_cijena = " 12,50 EUR "
izvorna_datoteka = " REZULTATI_2026.CSV "

oznaka = izvorna_oznaka.strip().upper().replace(" ", "-")
ime = izvorno_ime.strip().title()
zapis_cijene = izvorna_jedinicna_cijena.strip().replace(" EUR", "").replace(",", ".")
cijena = float(zapis_cijene)
datoteka = izvorna_datoteka.strip().lower().replace("_", "-")

print("Izvorna oznaka: [" + izvorna_oznaka + "]")
print("Uređena oznaka:", oznaka)
print("Broj crtica:", oznaka.count("-"))
print("Završna oznaka sadrži samo znamenke:", oznaka[-3:].isdigit())
print("Ime:", ime)
print("Jedinična cijena:", cijena)
print("Ukupan iznos za tri proizvoda:", cijena * 3)
print("Datoteka:", datoteka)
print("Počinje s rezultatima:", datoteka.startswith("rezultati"))
print("Završava s .csv:", datoteka.endswith(".csv"))
print("Položaj godine:", datoteka.find("2026"))

```

```

Izvorna oznaka: [ st 2026 015 ]
Uređena oznaka: ST-2026-015
Broj crtica: 2
Završna oznaka sadrži samo znamenke: True
Ime: Ana Anić
Jedinična cijena: 12.5
Ukupan iznos za tri proizvoda: 37.5
Datoteka: rezultati-2026.csv
Počinje s rezultatima: True
Završava s .csv: True
Položaj godine: 10

```

9 Oblikovanje ispisa pomoću f-stringova

9.1 Umetanje vrijednosti varijabli u tekst

Dosad smo vrijednosti ispisivali kao odvojene argumente funkcije `print()` ili smo tekstove spajali operatorom `+`. Takvi zapisi postaju nepregledni kada poruka sadrži više varijabli, a pri spajanju teksta i broja moramo paziti i na pretvorbu tipova.

F-string omogućuje da tekst poruke i vrijednosti varijabli zapišemo kao jednu cjelinu. Ispred početnog navodnika dodajemo slovo `f`, a imena varijabli ili izraze koje želimo umetnuti zapisujemo unutar vitičastih zagrada `{}`. Python pri izvođenju izračuna sadržaj zagrada i njegovo mjesto zamijeni tekstualnim prikazom dobivene vrijednosti.

Tekst izvan vitičastih zagrada ispisuje se doslovno. Zbog toga u istoj poruci možemo jasno vidjeti koji je dio stalni opis, a koji će se promijeniti ovisno o vrijednostima varijabli.

```
ime = "Ana"
bodovi = 81
print(f"Studentica {ime} ostvarila je {bodovi} bodova.")
print("Studentica {ime} ostvarila je {bodovi} bodova.")
```

▼ Prikaži očekivani ispis

```
Studentica Ana ostvarila je 81 bodova.
Studentica {ime} ostvarila je {bodovi} bodova.
```

Bez slova `f` dobivamo običan tekst i imena u vitičastim zgradama ne zamjenjuju se vrijednostima. U f-stringu Python sam izrađuje tekstualni prikaz vrijednosti, pa broj ne moramo ručno pretvarati pomoću `str()`.

9.2 Prikaz brojeva na dvije decimale

U rezultatima računanja broj često želimo prikazati u dogovorenom obliku. Cijene se, primjerice, uobičajeno prikazuju s dvije decimale, čak i kada je druga decimala nula. Broj `12.5` i prikaz `12.50` predstavljaju istu brojčanu vrijednost, ali drugi je prikladniji za izvještaj o cijeni.

Unutar f-stringa način prikaza navodimo iza dvotočke. Zapis `.2f` znači da broj želimo prikazati u fiksnom decimalnom obliku s dvije znamenke nakon decimalne točke. Time oblikujemo samo tekstualni prikaz, izvorna brojčana varijabla ostaje nepromijenjena.

```
iznos = 12.5
poruka = f"Iznos: {iznos:.2f} EUR"
print(poruka)
print(iznos)
print(type(poruka))
print(type(iznos))
```

▼ Prikaži očekivani ispis

```
Iznos: 12.50 EUR
12.5
<class 'str'>
<class 'float'>
```

U zapisu `{iznos:.2f}` dio prije dvotočke određuje vrijednost, a `.2f` traži prikaz u fiksnom decimalnom obliku s dvije decimale. Dobivena poruka je tekst. Broj `iznos` ostaje broj 12.5, pa ga i dalje možemo koristiti u računu. Dvotočka ovdje pripada pravilima oblikovanja i nema značenje raspona kao u `tekst[0:2]`.

Zadatak

Student Marko ostvario je 73.5 od mogućih 80 bodova. Kako biste izračunali postotak i jednim f-stringom ispisali poruku `Marko: 73.5/80 bodova (91.9 %)`? Postotak prikažite na jednu decimalu.

▼ Prikaži moguće rješenje i ispis

```
ime = "Marko"
ostvoreni_bodovi = 73.5
maksimalni_bodovi = 80
postotak = ostvoreni_bodovi / maksimalni_bodovi * 100

print(f"{ime}: {ostvoreni_bodovi}/{maksimalni_bodovi} bodova ({postotak:.1f} %)")
```

Marko: 73.5/80 bodova (91.9 %)

10 Usporedbe i logički izrazi

10.1 Operatori dodjele i usporedbe

Programi često moraju provjeriti odnos između vrijednosti: je li ostvaren dovoljan broj bodova, pripada li student određenoj grupi ili razlikuju li se dva zapisa. Rezultat takve provjere nije nova brojčana ili tekstualna vrijednost, nego logička vrijednost `True` ili `False`.

U prethodnoj vježbi već smo provjeravali uvjete u tabličnom kalkulatoru. Python također može usporediti dvije vrijednosti i vratiti `True` ili `False`. Posebno je važno razlikovati dodjelu `=` od usporedbe jednakosti `==`.

💡 Logički izraz

Logički izraz u primjerima uspoređuje vrijednosti ili povezuje provjere te daje logički rezultat. `bodovi = 60` dodjeljuje vrijednost, dok `bodovi == 60` provjerava je li postojeća vrijednost jednaka 60.

Operator	Značenje
<code>==</code>	jednako
<code>!=</code>	nije jednako
<code>></code>	veće od
<code>>=</code>	veće od ili jednako
<code><</code>	manje od
<code><=</code>	manje od ili jednako

```
bodovi = 60
grupa = "A"
print(bodovi == 60)
print(bodovi > 60)
print(bodovi >= 60)
print(grupa == "A")
print(grupa != "A")
```

▼ Prikaži očekivani ispis

True
False
True
True
False

Razlika između `>` i `>=` vidi se upravo na granici 60. Kada zadatak kaže „najmanje 60”, uključujemo i 60. To treba provjeriti testiranjem granične vrijednosti, ne samo primjera daleko iznad praga.

10.2 Usporedba tekstova i ujednačavanje njihova zapisa

Tekst se uspoređuje znak po znak. Velika i mala slova te razmaci dio su sadržaja, pa su Pythonu `"A"` i `"a"` različite vrijednosti, jednako kao `"Pula"` i `"Pula "`. Zbog toga dva zapisa koja čovjek protumači kao isti podatak mogu dati rezultat usporedbe `False`.

Ako pravila zadatka kažu da razlike u veličini slova i rubnim razmacima nisu važne, tekst prije usporedbe trebamo ujednačiti. Takva priprema omogućuje da uspoređujemo značenje podataka umjesto slučajnih razlika u zapisu.

```
unos_grupe = " a "  
print(unos_grupe == "A")  
grupa = unos_grupe.strip().upper()  
print(grupa == "A")
```

▼ Prikaži očekivani ispis

```
False  
True
```

Takvo ujednačavanje zapisa zovemo normalizacijom. Primjereno je ako pravila kažu da velika i mala slova ne razlikuju grupe. Ne treba automatski normalizirati svaki podatak: primjerice, sadržaj u kojem veličina slova nosi značenje mora se očuvati.

Tekstualni broj prije brojčane usporedbe treba pretvoriti u broj. Izraz `"12" == 12` vraća `False`. Usporedba dvaju tekstova poput `"10" < "2"` vraća `True` zbog poretka znakova, a ne zbog veličine brojeva. Zato je provjera tipa nužna prije računanja i uspoređivanja količina.

10.3 Provjera sadržavanja teksta operatorima `in` i `not in`

Operatori usporedbe poput `==` provjeravaju odgovaraju li dvije cijele vrijednosti. Ponekad nas zanima samo pojavljuje li se kraći tekst unutar duljeg, primjerice nalazi li se riječ `"podatkovne"` u nazivu kolegija. Za takvu provjeru koristimo operator `in`.

Izraz `trazeni_dio in tekst` vraća `True` ako se traženi niz znakova pojavljuje bilo gdje u tekstu. Operator `not in` provjerava suprotno: vraća `True` kada traženog dijela nema.

```
naziv = "osnove podatkovne znanosti"  
print("podatkovne" in naziv)  
print("Python" in naziv)  
print("Python" not in naziv)
```

▼ Prikaži očekivani ispis

True
False
True

Operator `in` provjerava postoji li traženi niz znakova bilo gdje u tekstu. Ne provjerava nužno cijelu riječ. Primjerice, `"ana" in "banana"` daje `True`. Operator `not in` provjerava odsutnost. Obje provjere razlikuju velika i mala slova.

10.4 Logički operatori and, or i not

Jedna usporedba odgovara na jedno pitanje, ali uvjeti u zadacima često ovise o više provjera. Student, primjerice, može trebati zadovoljiti i prag bodova i minimalan broj dolazaka. Logički operatori omogućuju da više rezultata `True` i `False` povežemo u jedan završni logički izraz.

U Pythonu koristimo operatore `and`, `or` i `not`, zapisane malim slovima. Za razliku od prethodne vježbe u tabličnom kalkulatoru, ovdje su to operatori, a ne funkcije koje pozivamo zapisom `AND(...)`. Rezultat povezivanja ponovno je logička vrijednost koju možemo spremići u varijablu ili ispisati.

Za logičke vrijednosti vrijedi:

Prvi uvjet	Drugi uvjet	and	or
True	True	True	True
True	False	False	True
False	True	False	True
False	False	False	False

`and` traži da oba uvjeta budu istinita. `or` traži da barem jedan bude istinit, uključujući mogućnost da su oba istinita. `not` okreće logičku vrijednost: od `True` dobivamo `False` i obratno. U ovoj vježbi povezujemo samo logičke vrijednosti i usporedbe.

```
dolasci = 4
bodovi = 72
rad_predan = False

uvjet_bodova_i_dolazaka = dolasci >= 3 and bodovi >= 60
potrebna_provjera = dolasci < 3 or not rad_predan

print(uvjet_bodova_i_dolazaka)
print(potrebna_provjera)
```

▼ Prikaži ispis i objašnjenje

True
True

Student zadovoljava uvjet dolazaka i bodova. Međutim, rad nije predan pa je `not rad_predan` istinito. Zato je i druga provjera istinita. Dvije varijable odgovaraju na različita pitanja i njihovi rezultati nisu proturječni.

Za složenije izraze koristimo zagrade radi čitljivosti. Uobičajeno je prvo izračunati pojedine jasno imenovane provjere, a zatim ih povezati. Za sada samo računamo i ispisujemo rezultat uvjeta. Kako na temelju uvjeta izvršiti različite naredbe, obradit ćemo u sljedećoj vježbi.

Zadatak

Potrebno je odrediti može li se aktivnost održati na otvorenom. Aktivnost se može održati samo ako su istodobno ispunjena sva tri uvjeta:

1. Temperatura je najmanje 18 °C
2. Temperatura nije viša od 28 °C
3. Ne pada kiša.

Za vrijednosti `temperatura = 24` i `pada_kisa = False` izračunajte rezultat svakog uvjeta zasebno. Zatim povežite sva tri rezultata u završni logički izraz koji pokazuje može li se aktivnost održati na otvorenom.

▼ Prikaži moguće rješenje i ispis

```
temperatura = 24
pada_kisa = False

dovoljno_toplo = temperatura >= 18
dovoljno_hladno = temperatura <= 28
vrijeme_suho = not pada_kisa
aktivnost_na_otvorenom = dovoljno_toplo and dovoljno_hladno and vrijeme_suho

print("Dovoljno toplo:", dovoljno_toplo)
print("Dovoljno hladno:", dovoljno_hladno)
print("Vrijeme suho:", vrijeme_suho)
print("Aktivnost na otvorenom:", aktivnost_na_otvorenom)
```

```
Dovoljno toplo: True
Dovoljno hladno: True
Vrijeme suho: True
Aktivnost na otvorenom: True
```

11 Primjer: obrada rezultata studenta

Dosad smo pojedine postupke učili odvojeno: uređivali smo tekst, pretvarali tipove podataka, računali, uspoređivali vrijednosti i oblikovali ispis. U stvarnom programu ti se koraci najčešće nadovezuju jedan na drugi. Sirovi podatak prvo pripremamo, zatim ga pretvaramo u odgovarajući tip, provodimo izračun ili provjeru i na kraju prikazujemo rezultat.

Sljedeći primjer povezuje gradivo prethodnih poglavlja u jedan cjelovit postupak. Važno je pratiti ne samo završni ispis nego i kako se vrijednost i njezin tip mijenjaju u svakom koraku.

11.1 Priprema podataka, izračun prosjeka i ispis rezultata

Zadatak: obradite rezultat jednog studenta. Ime i grupa imaju neujednačen zapis, a bodovi su primljeni kao tekst. Izračunajte prosjek i provjerite je li student ostvario prosjek od najmanje 60 bodova uz najmanje tri dolaska.

Izvorni zapis studenta:

Podatak	Zapisana vrijednost
Ime	" Ana Anić "
Grupa	" a "
Bodovi na prvom zadatku	"78"
Bodovi na drugom zadatku	"84"
Broj dolazaka	4

Postupak:

1. Zapišite izvorne vrijednosti u zasebne varijable
2. Uklonite rubne razmake iz imena i ujednačite oznaku grupe
3. Pretvorite bodove u cijele brojeve
4. Izračunajte prosjek dvaju zadataka
5. Uvjet za upis ocjene je prosjek zadataka veći od 50% te 3 ili više dolazaka. Je li uvjet zadovoljen?
6. Izradite čitljiv ispis s prosjekom na dvije decimale.

▼ Prikaži kod, očekivani izvještaj i objašnjenje

```
izvorno_ime = " Ana Anić "  
izvorna_grupa = " a "  
izvorni_prvi = "78"  
izvorni_drugi = "84"  
dolasci = 4  
  
ime = izvorno_ime.strip()  
grupa = izvorna_grupa.strip().upper()  
prvi_zadatak = int(izvorni_prvi)  
drugi_zadatak = int(izvorni_drugi)  
  
prosjek = (prvi_zadatak + drugi_zadatak) / 2  
uvjet_zadovoljen = prosjek >= 60 and dolasci >= 3  
  
print(f"Student: {ime}")  
print(f"Grupa: {grupa}")  
print(f"Prosjek: {prosjek:.2f}")  
print(f"Uvjet zadovoljen: {uvjet_zadovoljen}")
```

Student: Ana Anić
Grupa: A
Prosjek: 81.00
Uvjet zadovoljen: True

Izvorne vrijednosti ostaju dostupne za provjeru. Čišćenje stvara nove tekstove, pretvorba daje brojeve, računanje daje prosjek, a usporedba daje logičku vrijednost. Završni ispis prikazuje rezultate tih različitih koraka.

12 Čitanje pogrešaka i provjera programa

Pogreške su očekivan dio razvoja programa. **Sintaksne pogreške** nastaju kada kod nije zapisan prema pravilima jezika, primjerice kada nedostaje navodnik ili zagrada. Python ih prepozna prije pravilnog izvođenja programa.

Pogreške tijekom izvođenja nastaju u sintaksno ispravnom programu kada Python naiđe na operaciju koju ne može izvršiti, primjerice zbrajanje nepodržanih tipova ili dijeljenje nulom. Obje vrste zaustavljaju program i Python ih opisuje porukom pogreške.

Logičke pogreške ne moraju zaustaviti izvođenje. Program se izvrši bez poruke pogreške, ali zbog pogrešno postavljenog izraza ili pogrešne pretpostavke daje netočan rezultat. Zato izostanak poruke pogreške ne znači nužno da je program ispravan.

Cilj nije pamtit svaku moguću poruku, nego naučiti sustavno pronaći mjesto problema, prepoznati vrste uključenih vrijednosti i provjeriti odgovara li dobiveni rezultat zadatku koji rješavamo.

12.1 Vrste pogrešaka i čitanje poruka pogreške

Poruka pogreške opisuje problem na koji je Python naišao. Pri čitanju ispisa najprije pogledajte posljednji redak s vrstom i opisom pogreške, a zatim broj retka u svojoj datoteci. Mjesto na kojem je problem otkriven ne mora uvijek biti mjesto gdje ste ga uzrokovali na primjer, nezatvoren navodnik ili zagrada može utjecati na sljedeće retke.

Vrsta	Tipičan uzrok u ovoj vježbi
<code>SyntaxError</code>	Nedostaje navodnik, zagrada ili je zapis naredbe neispravan.
<code>NameError</code>	Ime nije definirano ili je pogrešno napisano.
<code>TypeError</code>	Operacija nije podržana za zadane tipove.
<code>ValueError</code>	Tip argumenta je prihvatljiv, ali sadržaj nije pogodan, primjerice <code>int("abc")</code> .
<code>ZeroDivisionError</code>	Dijeljenje nulom.
<code>IndexError</code>	Pristup nepostojećem indeksu.

Točan tekst poruke može se razlikovati među verzijama Pythona. Važno je razumjeti vrstu problema i vrijednosti koje su dovele do njega, a ne pamtit cijelu poruku.

12.2 Ispravljanje pogreške pri zbrajanju teksta i broja

Pri ispravljanju programa najprije trebamo utvrditi što smo željeli postići. Isti operator može imati različito značenje za različite tipove podataka, pa nije dovoljno samo ukloniti poruku pogreške, ispravak mora dati i smislen rezultat.

Ovaj program namjerno sadrži pogrešku:

```
bodovi = "78"
ukupno = bodovi + 2
print(ukupno)
```

Python ne može zbrojiti `str` i `int` tim operatorom. Problem se ne rješava samo premještanjem navodnika. Najprije odlučujemo želimo li računati brojčani zbroj ili spajati tekst. Ako želimo rezultat 80, tekst bodova treba pretvoriti u broj.

▼ Prikaži ispravak

```
bodovi = "78"
ukupno = int(bodovi) + 2
print(ukupno)
```

80

Postupak provjere kada program ne daje očekivano:

1. Provjerite pokrećete li spremljenu verziju prave datoteke
2. Pronađite redak na kojem se problem javlja
3. Privremeno ispišite ulaznu vrijednost i njezin tip
4. Provjerite redoslijed dodjela i operatora
5. Napravite jednu izmjenu pa ponovno pokrenite program
6. Nakon ispravka provjerite i granične primjere, a ne samo početni unos.

Ako zapnete tijekom samostalnog rada, javite se putem Google Chata ili e-maila. Priložite datoteku s kodom, cijelu poruku pogreške i kratak opis očekivanog rezultata kako bi se problem mogao ponoviti i objasniti.

Zadatak

Sljedeći program se izvršava bez poruke pogreške, ali ne daje očekivani ukupni iznos. Što program ispisuje, zašto je rezultat pogrešan i kako biste ga ispravili?

```
cijena = "12.50"
kolicina = 4
ukupno = cijena * kolicina
print("Ukupno:", ukupno)
```

▼ Prikaži objašnjenje i moguće rješenje

Operator `*` između teksta i cijelog broja ponavlja tekst, pa program ispisuje `12.5012.5012.5012.50`. Cijenu prije računanja treba pretvoriti u broj.

```
cijena = float("12.50")
kolicina = 4
ukupno = cijena * kolicina
print("Ukupno:", ukupno)
```

Ukupno: 50.0

13 Pregled svih funkcija i metoda

Ovaj pregled objedinjuje sve funkcije i metode korištene u skripti. Funkcije su navedene odvojeno od metoda tipa `str`, a unutar svake tablice poredane su abecedno.

13.1 Funkcije

Funkcija	Namjena
<code>bool()</code>	Pretvara vrijednost u <code>True</code> ili <code>False</code> prema pravilima istinitosti.
<code>float()</code>	Pretvara prikladnu vrijednost u decimalni broj tipa <code>float</code> .
<code>input()</code>	Prikazuje poruku i vraća korisnički unos kao <code>str</code> .
<code>int()</code>	Pretvara prikladnu vrijednost u cijeli broj tipa <code>int</code> .
<code>len()</code>	Vraća broj znakova u tekstualnoj vrijednosti.
<code>print()</code>	Ispisuje vrijednosti i poruke u terminal.
<code>round()</code>	Vraća broj zaokružen na zadani broj decimalnih mjesta.
<code>str()</code>	Stvara tekstualni prikaz vrijednosti.
<code>type()</code>	Vraća tip predane vrijednosti.

13.2 Metode tipa str

Metoda	Namjena
<code>capitalize()</code>	Povećava prvi znak teksta, a ostala slova pretvara u mala.
<code>count()</code>	Vraća broj nepreklapajućih pojavljivanja zadanog dijela teksta.
<code>endswith()</code>	Provjerava završava li tekst zadanim dijelom.
<code>find()</code>	Vraća indeks prvog pojavljivanja ili <code>-1</code> ako traženog dijela nema.
<code>isdigit()</code>	Provjerava je li neprazan tekst sastavljen od znamenki.

Metoda	Namjena
<code>lower()</code>	Vraća tekst sa slovima pretvorenima u mala.
<code>lstrip()</code>	Uklanja bjeline s lijevog ruba teksta.
<code>replace()</code>	Vraća tekst u kojem je zadani dio zamijenjen drugim tekstom.
<code>rstrip()</code>	Uklanja bjeline s desnog ruba teksta.
<code>startswith()</code>	Provjerava započinje li tekst zadanim dijelom.
<code>strip()</code>	Uklanja bjeline s oba ruba teksta.
<code>title()</code>	Povećava prvo slovo svake riječi, a ostala slova pretvara u mala.
<code>upper()</code>	Vraća tekst sa slovima pretvorenima u velika.

14 Samostalni zadaci

Svaki zadatak spremite u zasebnu `.py` datoteku, primjerice `zadatak_01.py`. U programima koristite opisna imena varijabli, a rezultate ispišite s jasnim oznakama. Svaki program mora raditi samostalno nakon novog pokretanja. Rješenja se nalaze u sljedećem poglavlju.

14.1 Zadatak 1

Za radionicu naziva `Obrada podataka` prijavljeno je 18 polaznika, predviđeno trajanje iznosi 1.5 sati, a prijave su još uvijek otvorene. Svaki podatak spremite u zasebnu varijablu i odaberite tip koji odgovara njegovu značenju. Program na kraju treba ispisati svaku spremljenu vrijednost i njezin tip kako biste mogli provjeriti svoj odabir.

14.2 Zadatak 2

Na početku prva grupa ima 14, a druga 17 studenata. Spremite broj studenata svake grupe u zasebnu varijablu i izračunajte početni ukupan broj studenata. Nakon toga prvoj se grupi pridružuju još dva studenta, pa ažurirajte njezin broj i ponovno izračunajte ukupan broj. Ispišite i početni i novi rezultat kako bi se jasno vidjela posljedica promjene.

14.3 Zadatak 3

Student je na tri provjere znanja ostvario 65, 80 i 92 boda. Svaki rezultat spremite zasebno, a zatim izračunajte ukupan broj bodova i njihov prosjek. Program treba ispisati oba rezultata, pri čemu prosjek treba biti prikazan s dvije decimale.

14.4 Zadatak 4

Trajanje aktivnosti zabilježeno je kao 367 sekundi. Pretvorite taj podatak u broj punih minuta i broj sekundi koje preostanu. Rezultat ispišite u jednoj razumljivoj poruci koja zasebno navodi minute i sekunde.

14.5 Zadatak 5

Za radionicu je potrebno kupiti osam komada materijala, a jedinična cijena jednog komada iznosi 3.75 eura. Na ukupan iznos kupnje odobrava se popust od 15 %. Izračunajte cijenu prije popusta, iznos odobrenog popusta i konačan iznos za plaćanje. Sva tri novčana iznosa ispišite s dvije decimale i jasnom oznakom što predstavljaju.

14.6 Zadatak 6

Napišite program koji od korisnika traži unos naziva proizvoda, njegove jedinične cijene i željene količine. Na temelju unesenih podataka izračunajte ukupnu vrijednost kupnje te je ispišite uz naziv proizvoda, prikazanu s dvije decimale. Pretpostavite da korisnik unosi ispravne vrijednosti, a program provjerite unosom proizvoda `Bilježnica`, jedinične cijene 2.40 eura i količine 5.

14.7 Zadatak 7

Oznaka grupe zapisana je kao " grupa b ", s nepotrebnim razmacima na rubovima i neujednačenim oblikom zapisa. Izradite novu vrijednost bez rubnih razmaka, sa svim slovima pretvorenima u velika i s crticom između riječi, tako da konačna oznaka glasi "GRUPA-B". Ispišite izvornu i obrađenu vrijednost unutar uglatih zagrada kako bi njihove granice i razmaci bili jasno vidljivi.

14.8 Zadatak 8

Identifikator "OPZ-2026-042" sastoji se od oznake kolegija, godine i završne troznamenaste oznake. Izdvojite svaki od tih dijelova u zasebnu varijablu i zatim ih pojedinačno ispišite. Završni dio treba ostati tekstualna vrijednost "042" kako bi početna nula bila sačuvana.

14.9 Zadatak 9

Jedinična cijena jednog proizvoda zapisana je kao tekst " 7,80 EUR ". Pripremite taj zapis za računanje i pretvorite ga u odgovarajuću brojčanu vrijednost. Zatim izračunajte ukupan iznos za tri proizvoda s tom jediničnom cijenom. Program treba ispisati izvorni zapis, dobivenu brojčanu cijenu, njezin tip i ukupan iznos prikazan s dvije decimale.

14.10 Zadatak 10

Naziv datoteke zapisan je kao " Rezultati_2026.CSV ". Pripremite novu vrijednost bez rubnih razmaka i u obliku koji omogućuje provjere neovisne o početnoj veličini slova. Zasebno provjerite završava li uređeni naziv nastavkom .csv, sadrži li tekst 2026 i započinje li tekstem rezultati. Ispišite uređeni naziv datoteke i rezultat svake od triju provjera.

14.11 Zadatak 11

Student je ostvario 58 bodova, zabilježena su mu četiri dolaska i predao je rad. Izračunajte prvi logički rezultat koji pokazuje ima li student istodobno najmanje 60 bodova i najmanje tri dolaska. Zatim izračunajte drugi rezultat koji pokazuje je li potrebna dodatna provjera zato što student ima manje od tri dolaska ili nije predao rad. Oba rezultata spremite zasebno i ispišite ih tako da jedan izračun ne zamijeni drugi.

14.12 Zadatak 12

Program treba izračunati prosjek dvaju rezultata te ispisati uređeni naziv grada. Popravite pogreške tako da ispis sadrži prosjek 75.0 i grad Pula bez rubnih razmaka.

```
prvi = "70"  
drugi = 80  
prosjek = prvi + drugi / 2  
grad = " Pula "  
grad.strip()  
print(Prosjek)  
print "[" + grad + "]"
```

Ispravite program i spremite funkcionalnu verziju u svoju datoteku. Uz izmijenjene retke dodajte komentare kojima ćete objasniti zašto je svaka promjena bila potrebna, a zatim provjerite daje li program traženi prosjek i pravilno uređeni naziv grada.

15 Rješenja samostalnih zadataka

Prikazana su moguća rješenja. Drukčija imena varijabli i drukčiji raspored koraka mogu biti ispravni ako program daje traženi rezultat i primjenjuje prikladne tipove. Ne uspoređujte samo završni tekst: provjerite kako je dobiven.

15.1 Rješenje zadatka 1

▼ Prikaži rješenje i ispis

```
naziv = "Obrada podataka"
broj_prijavljenih = 18
trajanje = 1.5
prijave_otvorene = True
print(naziv, type(naziv))
print(broj_prijavljenih, type(broj_prijavljenih))
print(trajanje, type(trajanje))
print(prijave_otvorene, type(prijave_otvorene))
```

```
Obrada podataka <class 'str'>
18 <class 'int'>
1.5 <class 'float'>
True <class 'bool'>
```

Otvorenost prijave opisuje stanje, pa smo je zapisali logičkom vrijednošću. Naziv je tekst, broj prijavljenih cijeli broj, a trajanje dopušta decimalni dio.

15.2 Rješenje zadatka 2

▼ Prikaži rješenje i ispis

```
prva_grupa = 14
druga_grupa = 17
pocetni_ukupno = prva_grupa + druga_grupa
prva_grupa += 2
novi_ukupno = prva_grupa + druga_grupa
print("Početno:", pocetni_ukupno)
print("Nakon promjene:", novi_ukupno)
```

```
Početno: 31
Nakon promjene: 33
```

Početni rezultat ostaje 31. Nakon promjene ulaznog podatka izračunali smo novi rezultat zasebnom naredbom.

15.3 Rješenje zadatka 3

▼ Prikaži rješenje i ispis

```
prvi = 65
drugi = 80
treći = 92
zbroj = prvi + drugi + treći
prosjeak = zbroj / 3
print("Zbroj:", zbroj)
print(f"Prosjeak: {prosjeak:.2f}")
```

Zbroj: 237

Prosjeak: 79.00

15.4 Rješenje zadatka 4

▼ Prikaži rješenje i ispis

```
trajanje = 367
minute = trajanje // 60
sekunde = trajanje % 60
print(f"Trajanje: {minute} minuta i {sekunde} sekundi.")
```

Trajanje: 6 minuta i 7 sekundi.

Šest potpunih minuta daje 360 sekundi. Ostatak je 7, pa provjera $6 * 60 + 7$ vraća početnih 367.

15.5 Rješenje zadatka 5

▼ Prikaži rješenje i ispis

```
kolicina = 8
cijena = 3.75
popust_postotak = 15
pocetni_iznos = kolicina * cijena
popust = pocetni_iznos * popust_postotak / 100
konacni_iznos = pocetni_iznos - popust
print(f"Početni iznos: {pocetni_iznos:.2f} EUR")
print(f"Popust: {popust:.2f} EUR")
print(f"Nakon popusta: {konacni_iznos:.2f} EUR")
```

Početni iznos: 30.00 EUR
Popust: 4.50 EUR
Nakon popusta: 25.50 EUR

15.6 Rješenje zadatka 6

▼ Prikaži rješenje i primjer unosa

```
naziv = input("Naziv proizvoda: ")
cijena = float(input("Jedinična cijena: "))
kolicina = int(input("Količina: "))
ukupno = cijena * kolicina
print(f"{naziv}: {ukupno:.2f} EUR")
```

Naziv proizvoda: Bilježnica
Jedinična cijena: 2.40
Količina: 5
Bilježnica: 12.00 EUR

Naziv ostaje tekst. Cijena se pretvara u `float`, a količina u `int`. Pretpostavili smo cijeli broj proizvoda i valjan brožčani unos.

15.7 Rješenje zadatka 7

▼ Prikaži rješenje i ispis

```
izvorno = " grupa b "  
uredjeno = izvorno.strip().upper().replace(" ", "-")  
print "[" + izvorno + "]"  
print "[" + uredjeno + "]"
```

[grupa b]
[GRUPA-B]

Najprije se uklanjaju rubni razmaci. Kada bismo sve razmake prvo zamijenili crticama, rubovi više ne bi bili razmaci i `strip()` ih ne bi uklonio. Redoslijed je važan.

15.8 Rješenje zadatka 8

▼ Prikaži rješenje i ispis

```
identifikator = "OPZ-2026-042"
kolegij = identifikator[:3]
godina = identifikator[4:8]
oznaka = identifikator[-3:]
print("Kolegij:", kolegij)
print("Godina:", godina)
print("Oznaka:", oznaka)
```

Kolegij: OPZ
Godina: 2026
Oznaka: 042

Oznaka ostaje tekst. Pretvorba u cijeli broj dala bi 42 i izgubili bismo početnu nulu u uobičajenom ispisu.

15.9 Rješenje zadatka 9

▼ Prikaži rješenje i ispis

```
izvorno = " 7,80 EUR "
tekst_cijene = izvorno.strip().replace(" EUR", "").replace(",", ".")
cijena = float(tekst_cijene)
ukupno = cijena * 3
print("Izvorno: [" + izvorno + "]")
print("Cijena:", cijena)
print(type(cijena))
print(f"Ukupno: {ukupno:.2f} EUR")
```

Izvorno: [7,80 EUR]
Cijena: 7.8
<class 'float'>
Ukupno: 23.40 EUR

15.10 Rješenje zadatka 10

▼ Prikaži rješenje i ispis

```
izvorno = " Rezultati_2026.CSV "
naziv = izvorno.strip().lower()
csv_nastavak = naziv.endswith(".csv")
sadrzi_godinu = "2026" in naziv
pocinje_rezultatima = naziv.startswith("rezultati")
print(naziv)
print("CSV nastavak:", csv_nastavak)
print("Sadrži godinu:", sadrzi_godinu)
print("Počinje oznakom rezultati:", pocinje_rezultatima)
```

rezultati_2026.csv
CSV nastavak: True
Sadrži godinu: True
Počinje oznakom rezultati: True

Provjeravamo tekst naziva, ne postojanje ili stvarni sadržaj datoteke. Nastavak `.csv` sam po sebi ne jamči da je sadržaj datoteke ispravan.

15.11 Rješenje zadatka 11

▼ Prikaži rješenje i ispis

```
bodovi = 58
dolasci = 4
rad_predan = True
uvjet_sudjelovanja = bodovi >= 60 and dolasci >= 3
dodatna_provjera = dolasci < 3 or not rad_predan
print("Uvjet sudjelovanja:", uvjet_sudjelovanja)
print("Dodatna provjera:", dodatna_provjera)
```

Uvjet sudjelovanja: False
Dodatna provjera: False

Prvi uvjet nije zadovoljen zbog bodova. Druga provjera ne odnosi se na bodove, nego samo na dolaske i predaju rada, pa je također `False`. Nemojte drugoj provjeri samostalno dodavati kriterij koji zadatak ne traži.

15.12 Rješenje zadatka 12

▼ Prikaži ispravak i objašnjenje

```
prvi = "70"
drugi = 80
# Tekst pretvaramo u broj; zbroj mora prethoditi dijeljenju.
prosijek = (int(prvi) + drugi) / 2
grad = " Pula "
# Metoda vraća novi tekst koji treba sačuvati.
grad = grad.strip()
# Koristimo točno ime varijable, s malim početnim slovom.
print(prosijek)
print "[" + grad + "]"
```

75.0
[Pula]

Bile su potrebne četiri sadržajne promjene: pretvorba tekstualnog broja, zagrade za prosjek, spremanje rezultata metode i usklađivanje naziva varijable. Program koji se izvršava bez poruke pogreške i dalje bi mogao dati pogrešan prosjek da smo propustili zagrade.